OAuth 2.0 PKCE

In the previous video, we learned OAuth 2.0 Authorization Code Flow.

Everything looked secure.

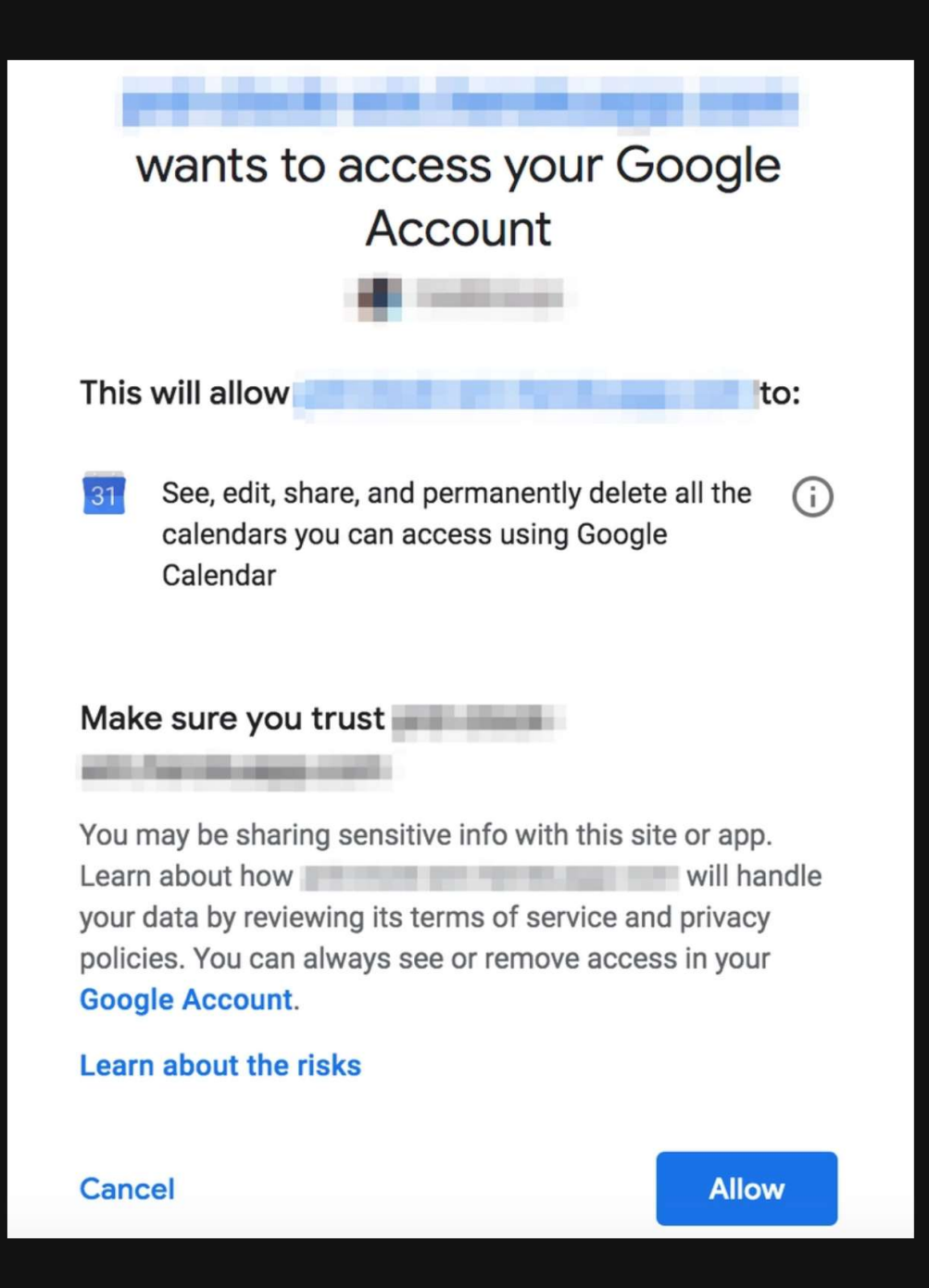
Browser never received the Access Token.

Backend securely exchanged the Authorization Code.

So why was PKCE invented?

Was OAuth broken?

Not exactly.



[redacted]
wants to access your Google Account
[redacted]

This will allow [redacted] to:

See, edit, share, and permanently delete all the calendars you can access using Google Calendar

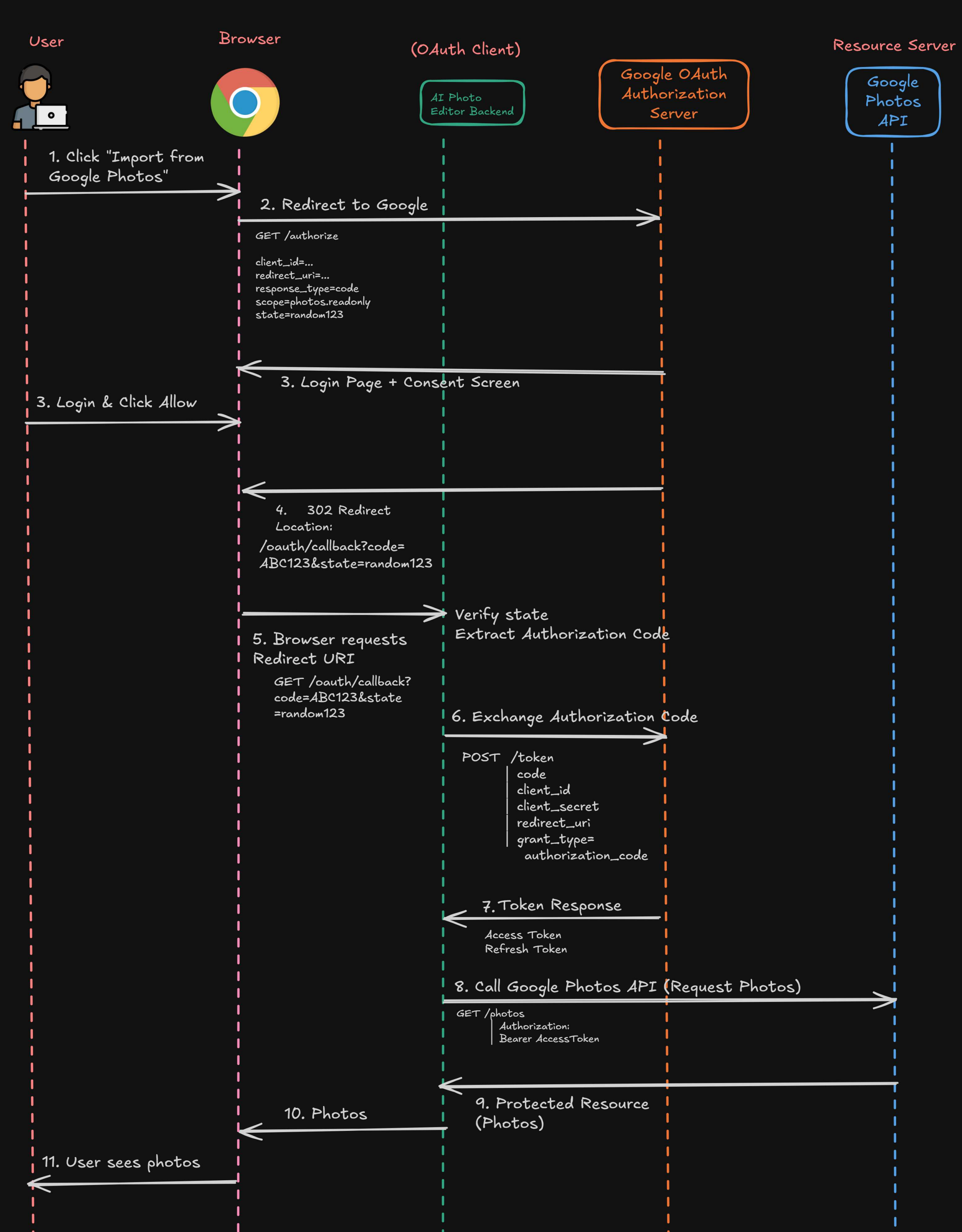
Make sure you trust [redacted]

You may be sharing sensitive info with this site or app. Learn about how [redacted] will handle your data by reviewing its terms of service and privacy policies. You can always see or remove access in your Google Account.

Learn about the risks

Cancel
Allow

Recap OAuth



User
Browser
(OAuth Client)
AI Photo Editor Backend
Google OAuth Authorization Server
Resource Server
Google Photos API
1. Click "Import from Google Photos"
2. Redirect to Google
GET /authorize
client_id=...
redirect_uri=...
response_type=code
scope=photos.readonly
state=random123
3. Login Page + Consent Screen
3. Login & Click Allow
4. 302 Redirect
Location:
/oauth/callback?code=ABC123&state=random123
5. Browser requests Redirect URI
GET /oauth/callback?code=ABC123&state=random123
Verify state
Extract Authorization Code
6. Exchange Authorization Code
POST /token
code
client_id
client_secret
redirect_uri
grant_type=authorization_code
7. Token Response
Access Token
Refresh Token
8. Call Google Photos API (Request Photos)
GET /photos
Authorization: Bearer AccessToken
9. Protected Resource (Photos)
10. Photos
11. User sees photos

Browser
-> Redirects User
-> Receives Authorization Code

Backend
-> Exchanges Code
-> Stores Tokens
-> Calls Google APIs

Google
-> Authenticates User
-> Issues Tokens

Google verifies

Client ID
Redirect URI
auth code

During token exchange

Backend also sends
Client Secret

Everything works.

Before OAuth Starts (Client Registration)

Before any OAuth request,

developer registers application with Google.

Google asks
Application Type?

Options

- Web Application
- Android
- iOS
- Desktop
- SPA

Google behaves differently depending on application type.

Confidential Client

Examples

- Spring Boot
- Node
- Django
- ASP.NET

Google gives Client ID
Client Secret

Why?

Because backend server can safely store secrets.
Nobody can download your server.
Nobody can inspect your memory.

Backend exchanges code

POST /token

client_id
client_secret
authorization_code

Google verifies

Client Secret

If correct
--> Access Token.

Public Client

Examples

- Android
- iPhone
- React
- Angular
- Vue
- Flutter

Will Google give
Client Secret ?

No.
Because users own these applications.

Anyone can

- Download APK
- Reverse engineer APK
- Open DevTools
- Read JavaScript
- Decompile desktop app

Therefore

Client Secret
--> Not Secret.

Google only gives

Client ID

The Problem

Backend proves identity using
Client Secret

Public Clients don't have one.

So how does Google know

that the application exchanging the Authorization Code
is really the same application that started OAuth?

This is the real problem.

Historical Problem

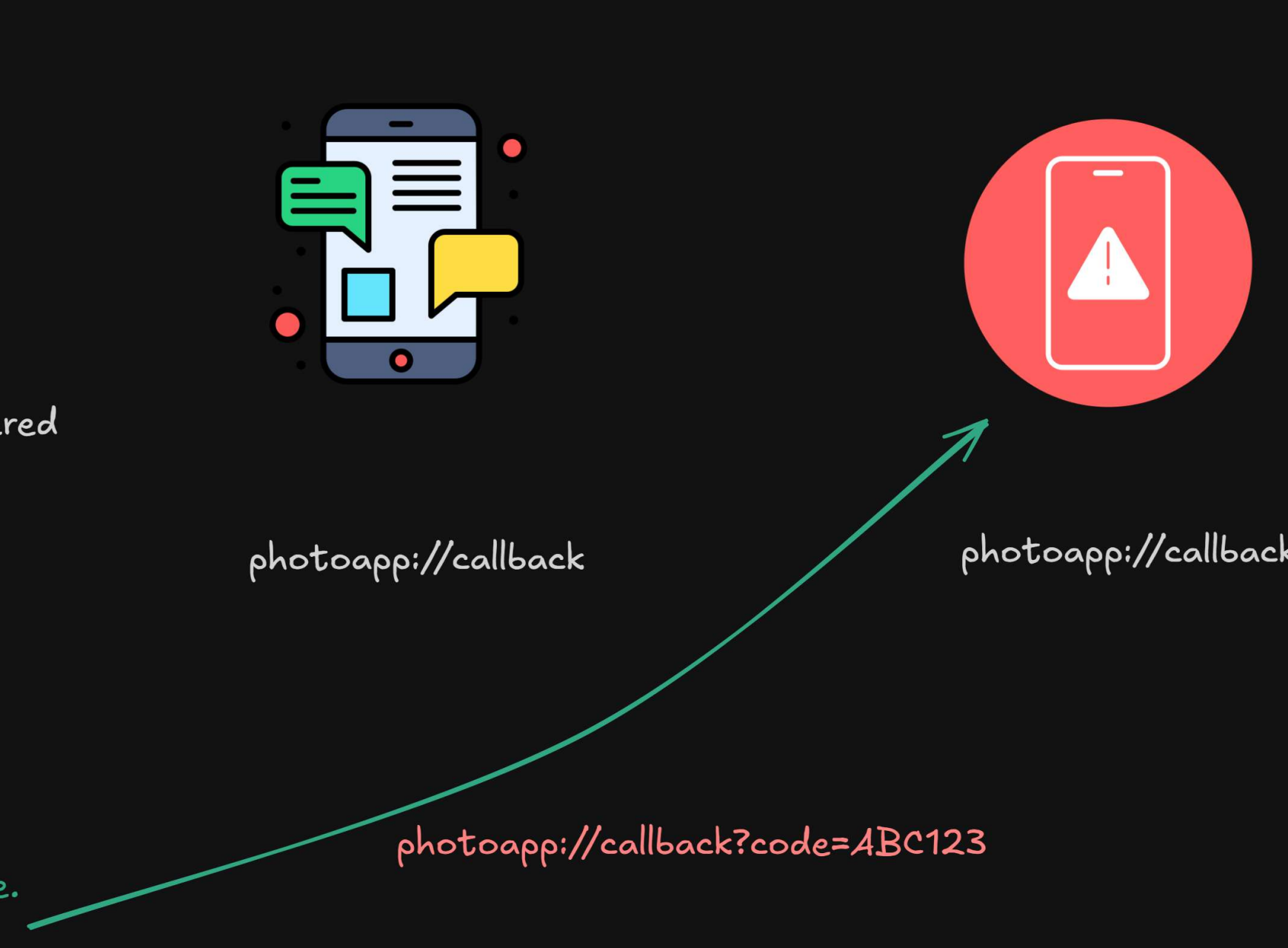
Years ago, Android used
Custom URI Schemes

Example

photoapp://callback

PhotoApp registered
photomagic://

Malicious App also registered
photomagic://



photoapp://callback
photoapp://callback
photoapp://callback?code=ABC123

User logs into Google.

Google redirects

Android OS receives redirect.
Android asks
Which app should open this?

Sometimes
Wrong app received Authorization Code

This is called
Authorization Code Interception Attack

PKCE was invented specifically to stop this attack.

Today

Android uses App Links.
iPhone uses Universal Links.

This attack is mostly solved.

Then
Why does PKCE still exist?

Because OAuth is
Platform Independent.
OAuth cannot depend on Android.

Tomorrow

New platform
New browser
New device
New attack.

OAuth needs protocol-level security.

Also
OAuth Security Best Current Practice recommends

Authorization Code Flow
PKCE

for all Public Clients.

Here comes PKCE

Instead of proving identity using
Client Secret

Public Clients prove possession using
Code Verifier

This is the biggest idea.

PKCE is basically
A one-time secret.

PKCE (Proof Key for Code Exchange)

Pronounced: "Pixy"

Now part of modern OAuth security recommendations and required by OAuth drafts and security best practices.

PKCE introduces only two new concepts.

1. code_verifier
2. code_challenge

Everything revolves around these.

code_verifier

This is
A cryptographically random string.

Example

j8s83jKSHD83jD83jKJHSD783jksd8sd7...

Properties

- generated by client
- random
- unique
- one-time
- never sent initially
- kept secret by client

Length

43-128 characters

Allowed characters

A-Z
a-z
0-9
-
.
_
~

Minimum entropy is important because attackers must not be able to guess it

code_challenge

Instead of sending verifier,

client computes
SHA256(code_verifier)

then
Base64URL Encode

Result
code_challenge

Formula

code_challenge

=

BASE64URL(
SHA256(code_verifier)
)

Why hash it?

Suppose Google received code_verifier
during first request.

Anyone intercepting that request could later exchange code.

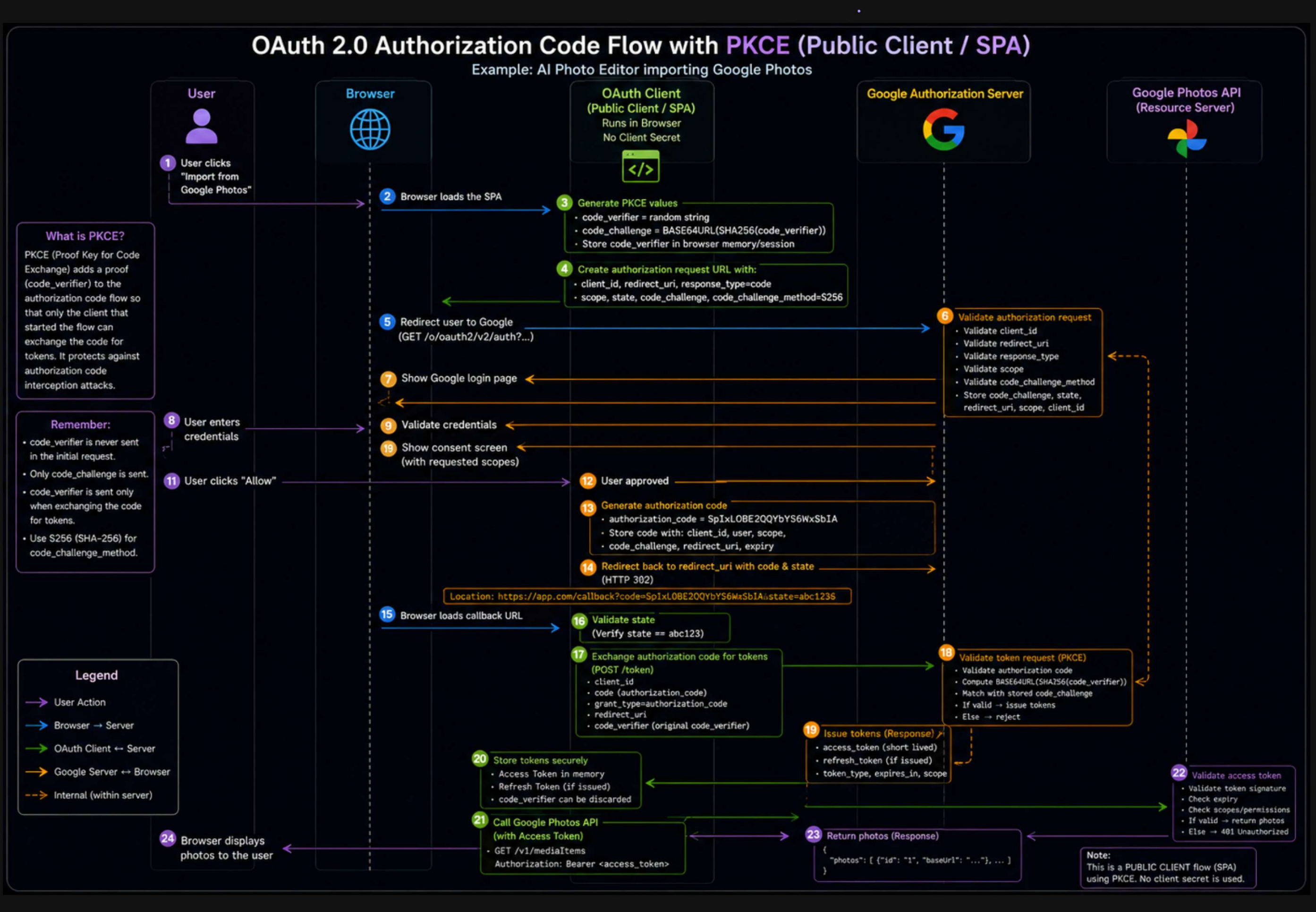
Instead,
Google only receives Hash(verifier)

Hash is one-way.

Google cannot recover original verifier.

Attacker also cannot.

Full Flow



OAuth 2.0 Authorization Code Flow with PKCE (Public Client / SPA)
Example: AI Photo Editor importing Google Photos
User
Browser
OAuth Client
(Public Client / SPA)
Runs in Browser
No Client Secret
Google Authorization Server
Google Photos API
(Resource Server)
What is PKCE?
PKCE (Proof Key for Code Exchange) adds a proof (code_verifier) to the authorization code flow so that only the client that started the flow can exchange the code for tokens. It protects against authorization code interception attacks.
Remember:
• code_verifier is never sent in the initial request.
• Only code_challenge is sent.
• code_verifier is sent only when exchanging the code for tokens.
• Use S256 (SHA-256) for code_challenge_method.
1 User clicks "Import from Google Photos"
2 Browser loads the SPA
3 Generate PKCE values
• code_verifier = random string
• code_challenge = BASE64URL(SHA256(code_verifier))
• Store code_verifier in browser memory/session
4 Create authorization request URL with:
• client_id, redirect_uri, response_type=code
• scope, state, code_challenge, code_challenge_method=S256
5 Redirect user to Google
(GET /o/oauth2/v2/auth?...)
6 Validate authorization request
• Validate client_id
• Validate redirect_uri
• Validate response_type
• Validate scope
• Validate code_challenge_method
• Store code_challenge, state, redirect_uri, scope, client_id
7 Show Google login page
8 User enters credentials
9 Validate credentials
10 Show consent screen
(with requested scopes)
11 User clicks "Allow"
12 User approved
13 Generate authorization code
• authorization_code = SpIxLOBE2QQYbYS6WxSbIA
• Store code with: client_id, user, scope,
• code_challenge, redirect_uri, expiry
14 Redirect back to redirect_uri with code & state
(HTTP 302)
Location: https://app.com/callback?code=SpIxLOBE2QQYbYS6WxSbIA&state=abc123
15 Browser loads callback URL
16 Validate state
(Verify state == abc123)
17 Exchange authorization code for tokens
(POST /token)
• client_id
• code (authorization_code)
• grant_type=authorization_code
• redirect_uri
• code_verifier (original code_verifier)
18 Validate token request (PKCE)
• Validate authorization code
• Compute BASE64URL(SHA256(code_verifier))
• Match with stored code_challenge
• If valid → issue tokens
• Else → reject
19 Issue tokens (Response)
• access_token (short lived)
• refresh_token (if issued)
• token_type, expires_in, scope
20 Store tokens securely
• Access Token in memory
• Refresh Token (if issued)
• code_verifier can be discarded
21 Call Google Photos API
(with Access Token)
• GET /v1/mediaItems
Authorization: Bearer <access_token>
22 Validate access token
• Validate token signature
• Check expiry
• Check scopes/permissions
• If valid → return photos
• Else → 401 Unauthorized
23 Return photos (Response)
{ "photos": [{"id": "1", "baseUrl": "..."}, ...] }
24 Browser displays photos to the user
Legend
User Action
Browser → Server
OAuth Client ↔ Server
Google Server ↔ Browser
Internal (within server)
Note:
This is a PUBLIC CLIENT flow (SPA) using PKCE. No client secret is used.

OAuth 2.0 Authorization Code Flow with PKCE (Public Client / SPA)

In this example, we have an AI Photo Editor web application that allows users to import photos from Google Photos.
Since the application is a Single Page Application (SPA) running entirely inside the browser,
it is considered a Public Client.
Unlike confidential clients, a public client cannot securely store a Client Secret.
Therefore, it uses PKCE (Proof Key for Code Exchange) to securely perform the Authorization Code Flow.

Participants:

- User — Wants to import Google Photos.
- Browser — Displays web pages and sends HTTP requests.
- OAuth Client (SPA/Public Client) — Your React/Angular/Vue application running inside the browser.
- Google Authorization Server — Authenticates users and issues OAuth tokens.
- Google Photos API (Resource Server) — Stores and serves the user's photos.

Step 1 — User clicks "Import from Google Photos"

The user opens the application and clicks the "Import from Google Photos" button.

Nothing has been sent to Google yet. This action simply tells the application to begin the OAuth authorization flow.

Step 2 — OAuth Client generates PKCE values

Before redirecting the user to Google, the OAuth Client generates a cryptographically random string called the code_verifier.

Example:

code_verifier = a8GkL9xT2pQrA92Kd8x...

The code_verifier is secret and must never be sent during the authorization request.

Next, the OAuth Client generates a code_challenge by hashing the code_verifier using SHA-256 and Base64URL encoding the result.

code_challenge = BASE64URL(SHA256(code_verifier))

Example:

code_challenge = kd89asD7qN4L...

Unlike the verifier, the code_challenge is safe to send to Google because it is only a one-way hash.

Finally, the OAuth Client stores the code_verifier temporarily in
browser memory (or another temporary browser storage mechanism).
It will be required later during the token exchange.

At this point:

Browser stores:
- code_verifier

Google knows:
- Nothing

Step 3 — OAuth Client creates the Authorization Request

The OAuth Client builds the authorization URL.

Example:

```
https://accounts.google.com/o/oauth2/v2/auth
client_id=YOUR_CLIENT_ID
redirect_uri=https://app.com/callback
response_type=code
scope=https://www.googleapis.com/auth/photoslibrary.readonly
state=abc123
code_challenge=kd89asD7...
code_challenge_method=S256
```

Notice something very important:

-> The request contains the code_challenge.

-> The request does NOT contain the code_verifier.

This is the key idea behind PKCE.

Step 4 — Browser redirects the user to Google

The browser sends the authorization request to Google's Authorization Endpoint.

The browser simply redirects the user to Google's login page.

Step 5 — Google validates the Authorization Request

Google validates:

- client_id
- redirect_uri
- response_type
- requested scopes
- code_challenge
- code_challenge_method

If everything is valid, Google stores:

- client_id
- redirect_uri
- requested scopes
- state
- code_challenge

Notice that Google still does NOT know the code_verifier.

Step 6 — Google shows the Login Page

Google sends its login page to the browser.

The browser displays the login screen where the user enters their Google email and password.

Step 7 — User enters credentials

The user enters their email and password.

The browser sends these credentials to Google.

Google verifies the user's identity.

Step 8 — Google displays the Consent Screen

After successful authentication, Google displays the consent screen.

Example:

AI Photo Editor wants permission to:

-> View your Google Photos

The user can either:

- Allow
- Cancel

Step 9 — User clicks "Allow"

The user approves the requested permissions.

The browser informs Google that the user has granted consent.

Step 10 — Google creates an Authorization Code

Google generates a short-lived Authorization Code.

Example:

authorization_code = SpIxLOBEZQQYbYS6WxSbIA

Google associates this authorization code with:

- client_id
- redirect_uri
- user
- requested scopes
- code_challenge
- expiration time

Google then redirects the browser back to the application's callback URL.

Example:

https://app.com/callback?code=SpIxLOBEZQQYbYS6WxSbIA&state=abc123

Notice once again:

Google still has never received the code_verifier.

Step 11 — Browser loads the Callback URL

The browser opens the callback URL.

Your SPA starts running again.

The OAuth Client extracts two values from the URL:

- authorization_code
- state

Step 12 — OAuth Client validates the State

Earlier, the OAuth Client generated:

state = abc123

The callback URL also contains:

state = abc123

The OAuth Client compares the two values.

If they match, the response is accepted.

If they do not match, the request is rejected.

This protects against Cross-Site Request Forgery (CSRF) attacks.

Step 13 — OAuth Client exchanges the Authorization Code for Tokens

The OAuth Client sends a POST request to Google's Token Endpoint.

The request includes:

- client_id
- authorization_code
- redirect_uri
- grant_type=authorization_code
- code_verifier

This is the FIRST time Google receives the code_verifier.

Step 14 — Google verifies PKCE

Google already has the stored code_challenge from the authorization request.

Now Google computes:

BASE64URL(SHA256(code_verifier))

It compares the computed value with the stored code_challenge.

If they match:

-> Google knows that the same client which started the authorization flow is now exchanging the authorization code.

If they do not match:

-> Google rejects the request and no tokens are issued.

This verification is the core security feature provided by PKCE.

Why PKCE is Secure

Suppose an attacker somehow steals the Authorization Code during the redirect.

The attacker now tries to exchange the authorization code for tokens.

Google asks for the corresponding code_verifier.

However, the attacker only has the Authorization Code.

The attacker does NOT know the secret code_verifier that was generated and stored by the original OAuth Client.

Without the correct code_verifier:

- Google cannot reproduce the stored code_challenge.
- PKCE verification fails.
- No tokens are issued.

This prevents Authorization Code Interception Attacks.

Step 15 — Google issues Tokens

If PKCE verification succeeds, Google returns:

- Access Token
- Refresh Token (if issued)
- Token Type
- Expiration Time
- Granted Scopes

The OAuth Client now has a valid Access Token.

Step 16 — OAuth Client stores Tokens

The OAuth Client stores the Access Token securely.

For browser-based applications, storing the Access Token in memory is generally preferred over long-term browser storage.

If a Refresh Token is issued, it should also be stored securely according to the application's security model.

The code_verifier is no longer required and can now be safely discarded.

Step 17 — OAuth Client calls Google Photos API

The OAuth Client calls the Google Photos API using the Access Token.

Example:

GET /v1/mediaItems

Authorization: Bearer ACCESS_TOKEN

The Access Token proves that the user has already granted permission.

Step 18 — Google Photos validates the Access Token

The Google Photos API validates:

- Token signature
- Token expiration
- Granted scopes
- Required permissions

If everything is valid, access is granted.

Otherwise, the request is rejected.

Step 19 — Google returns Photos

Google Photos API returns the requested photos to the OAuth Client.

Step 20 — Browser displays Photos

The OAuth Client receives the photos and displays them inside the application.

The user can now browse and import their Google Photos.

Complete Flow Summary

1. User clicks "Import from Google Photos."
2. OAuth Client generates a code_verifier and a code_challenge.
3. OAuth Client stores the code_verifier locally and sends only the code_challenge to Google.
4. Browser redirects the user to Google.
5. Google validates the authorization request and stores the code_challenge.
6. Google displays the login page.
7. User logs in.
8. Google displays the consent screen.
9. User grants permission.
10. Google generates an Authorization Code and redirects the browser back to the application.
11. OAuth Client extracts the Authorization Code and validates the state parameter.
12. OAuth Client sends the Authorization Code and the original code_verifier to Google's Token Endpoint.
13. Google hashes the received code_verifier and compares it with the stored code_challenge.
14. If they match, Google issues an Access Token (and optionally a Refresh Token).
15. OAuth Client stores the Access Token securely.
16. OAuth Client calls the Google Photos API using the Access Token.
17. Google Photos validates the token.
18. Google returns the user's photos.
19. The browser displays the photos to the user.

The OAuth 2.0 Authorization Code Flow with PKCE is now complete.

PKCE replaces the role of the Client Secret for public clients during the token exchange.
Instead of proving identity with a long-lived shared secret,
the client proves possession of a fresh, one-time Code Verifier generated for that authorization request